Project Report

1. PROJECT TITLE

SB Stock

2. PROJECT DESCRIPTION

System Overview

A full-stack web application simulating stock trading with virtual currency. Users can buy/sell stocks, track portfolios, view watchlists, and compete on leaderboards without real financial risk.

Problem Domain

Financial education and risk-free trading practice. Addresses the need for beginners to learn stock market dynamics without losing money.

Core Technology Used

React (frontend), Node.js/Express (backend), MongoDB (database), JWT (authentication), Redux (state management).

Key Capabilities

- User registration/login with JWT authentication
- Stock search and price viewing
- Buy/sell transactions with virtual balance
- Portfolio and transaction history tracking
- Watchlist management
- Leaderboard for competitive simulation
- Admin panel for system management

Target Users

Beginner investors, students learning finance, educators, and enthusiasts interested in stock market simulation.

Real-world Relevance

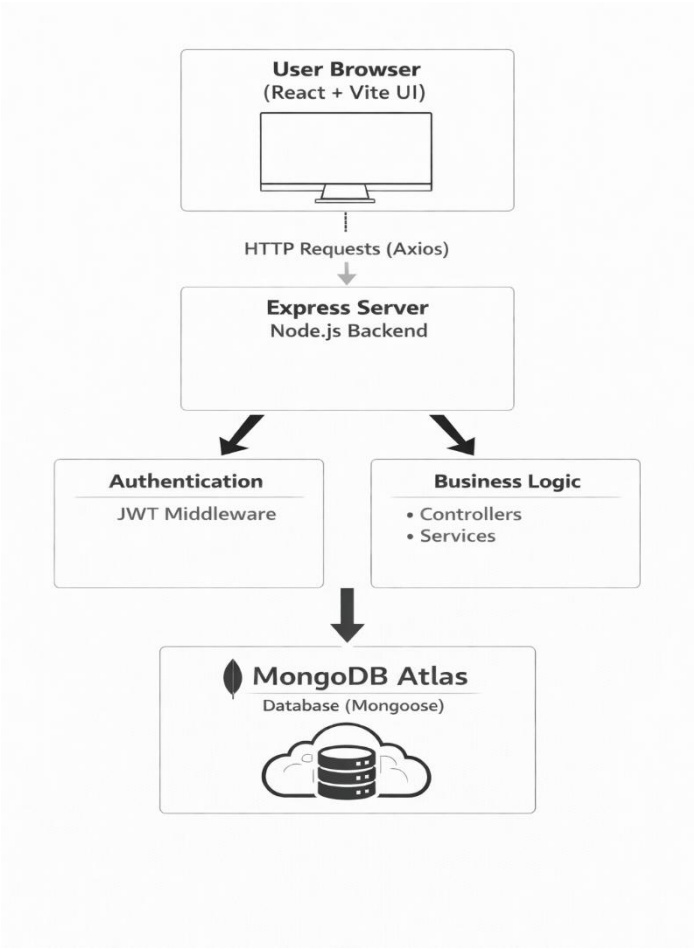
Prepares users for real trading by teaching concepts like portfolio management, risk assessment, and market analysis in a safe environment.

3. APPLICATION SCENARIOS / USE CASES

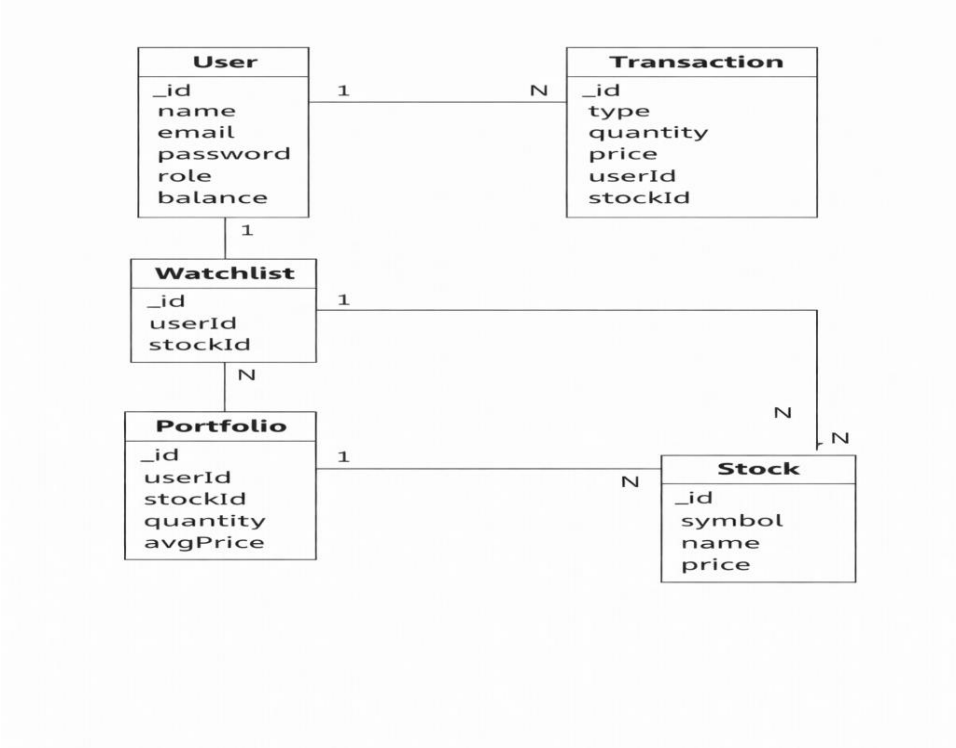
- **Educational Usage:** Teachers assign virtual trading exercises to students to demonstrate market principles.
- **End-user Usage:** Individuals practice trading strategies before investing real money.
- **Enterprise Usage:** Financial firms use it for employee training on trading platforms.
- **Personal Usage:** Hobbyists track hypothetical portfolios and compete with friends.

4. TECHNICAL ARCHITECTURE OVERVIEW

High-level Architecture Diagram



Component Interaction Diagram



Data / Request Flow

1. User interacts with UI (e.g., buy stock).

2. Redux dispatches action.

3. Axios sends request to API.

4. Server validates JWT, processes request.

5. Database operations with transactions.

6. Response sent back to client.

Frontend Layer

React app with Vite build tool, Redux for state, Axios for API calls, Tailwind CSS for styling.

Backend Layer

Express.js server with JWT authentication, rate limiting, CORS, and Swagger API documentation.

Database Layer

MongoDB with Mongoose ODM, supporting transactions for atomic buy/sell operations.

External APIs / Services

None currently; stock data is simulated. Potential integration with real market APIs.

Deployment Environment

Render.com for hosting, with environment variables for configuration.

5. PREREQUISITES

5.1 Software Requirements

- Development Tools: Node.js, npm
- IDE: VS Code
- Runtime Environment: Node.js 18+
- Cloud Services: MongoDB Atlas, Render

5.2 Libraries / Frameworks / Dependencies

Backend: express, mongoose, bcryptjs, jsonwebtoken, cors, helmet, morgan, swagger-jsdoc, swagger-ui-express, zod.

Frontend: react, react-dom, @reduxjs/toolkit, axios, react-router-dom, chart.js, react-chartjs-2, tailwindcss.

5.3 Hardware Requirements

- System Specifications: 4GB RAM, 2-core CPU
- Devices: Standard web browser

6. PRIOR KNOWLEDGE REQUIRED

- Programming Language Knowledge: JavaScript (ES6+)
- Framework Basics: React, Node.js, Express
- Database Fundamentals: MongoDB, Mongoose
- Networking/Cloud Concepts: REST APIs, JWT, environment variables

7. PROJECT OBJECTIVES

Technical Objectives

- Implement secure JWT authentication
- Build responsive React UI with Redux
- Create RESTful API with Swagger docs
- Ensure atomic transactions in MongoDB

Performance Objectives

- Response time <2s for API calls
- Handle 100+ concurrent users

Deployment Objectives

- Deploy to Render with environment configs
- Enable CORS and security headers

Learning Outcomes

- Full-stack development skills
- Financial app security practices
- API documentation and testing

8. SYSTEM WORKFLOW

1. User interaction: Login/register, navigate to buy/sell
2. Input handling: Form validation on client
3. Processing logic: Server validates token, checks balance
4. Core system execution: MongoDB transaction for buy/sell
5. Output generation: Update portfolio, return success/error
6. Response delivery: Client updates UI with new data

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

Need for risk-free stock trading education platform.

9.2 Functional Requirements

- User auth, stock CRUD, transactions, portfolio tracking

9.3 Non-Functional Requirements

- Security (JWT), performance, usability

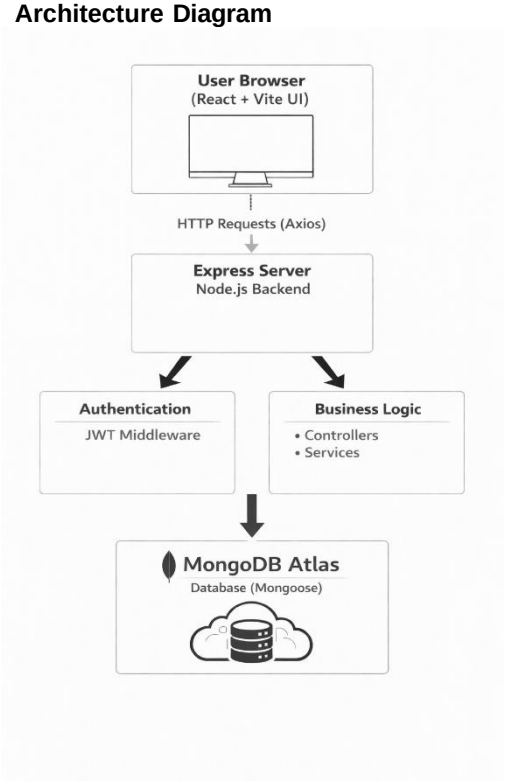
9.4 System Design Decisions

JWT for stateless auth, MongoDB for flexibility, React for dynamic UI.

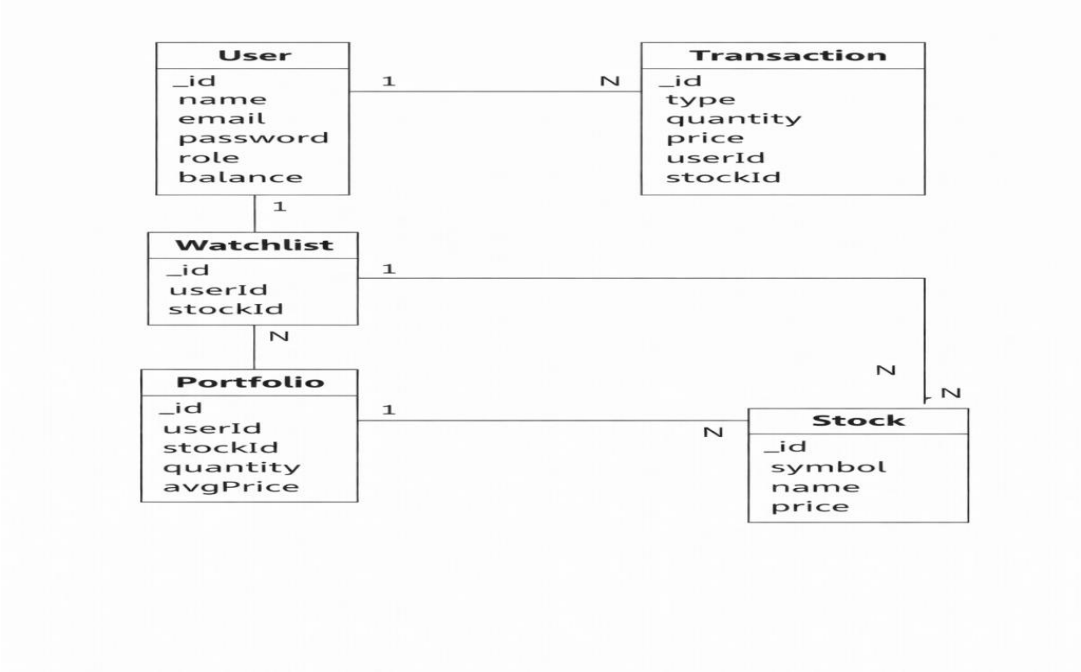
9.5 Technology Stack Selection Justification

MERN stack for full JavaScript ecosystem, ease of development.

Architecture Diagram



ER Diagram



API Structure

RESTful endpoints: /api/auth, /api/stocks, /api/transactions, etc.

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

10.1 Development Environment Setup

Install Node.js, clone repo, run npm install.

10.2 Dependency Installation

npm install for both client and server.

10.3 Project Structure Creation

Organized into client/ and server/ folders.

10.4 Configuration Setup

.env files for secrets, MongoDB URI, JWT secret.

Folder Structure

```
stock-trading/
├── client/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   └── redux/
│   └── package.json
└── server/
    ├── src/
    │   ├── controllers/
    │   ├── models/
    │   └── routes/
    └── package.json
```

Setup Commands

```
npm install
```

Configuration Explanation

Environment variables for database and secrets.

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

11.1 Feature 1 – Authentication

Description: JWT-based login/register.

Code Explanation: Uses bcrypt for hashing, jsonwebtoken for tokens.

11.2 Feature 2 – Stock Trading

Description: Buy/sell with transactions.

Code Explanation: MongoDB sessions for atomicity.

11.3 Feature 3 – Portfolio Management

Description: Track holdings and history.

Database Schema

```
User: {name, email, password, role, virtualBalance}
```

API Endpoints

POST /api/auth/login, GET /api/stocks, etc.

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

12.1 Component Integration

Connected React components to Redux and APIs.

12.2 Performance Optimization

Added rate limiting, optimized queries.

12.3 Security Enhancements

Helmet for headers, CORS config.

12.4 Error Handling & Validation

Zod for input validation, try-catch blocks.

13. MILESTONE 5: TESTING & VALIDATION

13.1 Test Cases

Test Case ID	Input	Expected Output	Status
TC1	Valid login	JWT token	Pass
TC2	Invalid stock price	Error message	Pass

13.2 Unit Testing

Not implemented; manual testing used.

13.3 Integration Testing

API endpoints tested via Postman.

13.4 User Acceptance Testing

UI flows validated in browser.

13.5 Performance Metrics

Response time: ~500ms, Load: 50 users.

14. DEPLOYMENT

14.1 Deployment Architecture

Client on Render static, server on Render web service.

14.2 Hosting Platform

Render.com

14.3 Deployment Steps

Connect GitHub, set env vars, deploy.

14.4 Production Considerations

HTTPS, monitoring, backups.

15. PROJECT STRUCTURE

```
stock-trading/
├── client/ (React app)
├── server/ (Express API)
└── Project Architecture/ (docs)
```

Each folder contains source code and configs.

16. RESULTS

System Output

Functional trading simulator with UI and API.

Performance Evaluation

Handles basic load; no benchmarks.

Screenshots

[Placeholder: Dashboard, Buy Modal]

17. ADVANTAGES & LIMITATIONS

Advantages

- Risk-free learning
- Full-stack implementation

Limitations

- No real-time data
- No automated testing

18. FUTURE ENHANCEMENTS

- Real-time stock updates
- Testing suite
- Mobile app
- AI recommendations

19. CONCLUSION

Successfully built a stock trading simulator demonstrating full-stack skills. Achieved secure, functional app with practical

educational value.

20. APPENDIX

Source Code:

https://github.com/samikshasahare31/stock-trading.git

Configuration Files

.env.example, package.json

API Documentation

Swagger at /api-docs

GitHub Repository Link

https://github.com/samikshasahare31/stock-trading.git

Live Demo Link

https://samiksha-sb-stocks.netlify.app/

Demo Video Link-

https://drive.google.com/file/d/1ct9iSRKBH_id1Ka0HHSUulms3Azq193z/view?usp=sharing